Paws Haven Academy

Manual 0

Welcome to Paws Haven

Your first week in the control room

Learn Python tools for data. Meet Mia. Help the animals.

This manual is for one explorer. It has 5 lessons and one big project. You do not need the internet for anything except Google Colab.

For the Parent or Coach

This manual is written so your child can work alone. Your job is small but important: keep them moving without handing over answers. Please read this page before the first session.

Before the first session

1. Colab needs a Google account. Children under 13 usually need a parent-managed (Family Link) account, so please set this up with them.
2. Open colab.research.google.com once to confirm it loads and can create a new notebook.
3. Pick two fixed times each week. The plan assumes about 2 hours a week.

Suggested schedule (about 2 weeks)

Week	Lessons	Time needed
Week 1	Lesson 1 (control room), Lesson 2 (reading errors), Lesson 3 (tables)	about 2 hours
Week 2	Lesson 4 (objects), Lesson 5 (the big project)	about 2 hours

How to help without giving answers

- Ask: **What did you expect? What happened? What does the last line of the error say?**
- Let them struggle for about 10 minutes before they climb the hint ladder (on the "How Each Lesson Works" page).
- Never type the fix for them. Point at a line, not at the answer.
- Ask for the 30-second explain-back after every lesson. If they cannot do it, repeat the lesson's Part A.

What to report back (the Progress Report page near the end)

After each lesson, please note: minutes taken, the highest hint rung used, whether the explain-back worked, a 1-5 confidence score, and any error that stuck them. When they finish Lesson 5, send their project code. With that I can adjust the next manual to fit what they actually need.

Welcome, Explorer!

You have just joined **Paws Haven**, a small animal rescue run by a kid named **Mia**. There are rabbits, cats, a hamster who escapes every Tuesday, and one very loud parrot named Kiwi.

Mia needs help. She is going to build programs that count animals, find patterns, and one day even *predict* things. But first she has to learn her tools. You will learn them with her.

What you will be able to do after this manual

- Use Google Colab to write and run Python code.
- Read an error message and fix the bug, like a detective.
- Turn a list of animals into a table and ask questions about it.
- Use objects with the dot (like `thing.action()`).
- Build a small data project from a blank page, all by yourself.

You already know variables, loops, functions, lists, and dictionaries. That is why this manual does not teach them again. It builds a bridge from what you know to the tools used for machine learning.

The 6 Rules of the Control Room

Mia's Rules

1. **Type it, do not paste it.** Your fingers learn what your eyes only see. (Exception: pasting is fine when a lesson says so.)
2. **Predict before you run.** Write your guess on paper first. Wrong guesses are the best moments, because that is when your brain grows.
3. **Break it on purpose.** You will always learn something when it crashes. Crashing is not failing.
4. **Struggle for 10 minutes** before you ask for help. Then use the hint ladder, one rung at a time.
5. **Explain it out loud** to a person, a pet, or a stuffed animal after every lesson.
6. **Do Part B without looking at the answers.** The answers are at the back. They are for checking, not for starting.

How Each Lesson Works

Part	What you do
The Story	Read about Mia and the problem she needs to solve. (2 minutes)
Part A: Copy and Run	Type the example and run it. Predict first. Then read what happened.
Part B: Your Turn	A similar task with a twist. No code is given. You do it yourself.
Break It on Purpose	Make the program crash and read the error.
Checkpoint Question	A prediction question. Guess first, then test it.
Stuck?	The most common problems and how to fix them.
Explain-Back	Tell someone the idea in 30 seconds.
Lesson Log	Write down how it went. This helps your coach.

The Hint Ladder

When you are stuck, climb down **one rung at a time**. Do not jump to rung 5. Each rung you skip is learning you skip.

Rung	What to do
1	Read the error message out loud. Say in your own words what it means.
2	Check the "Stuck?" box for that lesson.
3	Read **Hint 1** in the Hints section at the back.
4	Read **Hint 2** (a bigger hint, sometimes part of the code shape).
5	Look at the full answer. Then close the manual and redo it from memory **tomorrow**.

The 10-minute rule: if you have been stuck for 10 minutes, move down a rung. If you have been stuck for 30 minutes, stop, take a break, and tell your coach. Being stuck for an hour teaches nothing.

Writing what you expect to happen

In every lesson you will see the word **Predict**. Keep a piece of paper next to you. Write the number of the step and your guess. After you run the code, circle **R** if you were right or **W** if you were wrong. Aim for lots of Ws early. Wrong predictions mean you found something your brain did not know yet.

Lesson 1: The Control Room

Time: about 30 minutes. Goal: open Colab and make it talk.

The Story

Mia just got the keys to Paws Haven: 12 rabbits, 7 cats, and one very loud parrot. In the back room is a **control room**, a computer where Mia will build programs to help the animals.

Today she walks in and flips the first switch. Your control room is called **Google Colab**. It is a notebook where you write code in boxes called **cells**. You run a cell and the answer appears right under it.

Part A: Copy and Run

1. Go to **colab.research.google.com** and click **New notebook**. (Sign in if it asks.)
2. Click the title at the top-left and rename it `paws_haven_0`.
3. Click inside the first cell and **type this by hand** (Rule 1):

```
shelter_name = "Paws Haven"
rabbits = 12
cats = 7
print("Welcome to", shelter_name)
print("Animals so far:", rabbits + cats)
```

Predict: What will the last line print? Write it on paper.

Run the cell: click the round play button on the left, or press **Shift + Enter**.

You should see:

```
Welcome to Paws Haven
Animals so far: 19
```

Colab tour (2 minutes)

- **+ Code** adds a new cell for code. **+ Text** adds a cell for notes.
- **Shift + Enter** runs the cell and jumps to the next one.
- **Runtime menu** has **Restart session** (wipes memory) and **Run all** (runs every cell from top to bottom).
- Colab saves your notebook in Google Drive. But variables are **forgotten** when the session resets. Re-run your cells from the top to get them back.
- When you type a quote or a bracket, Colab may add the closing one for you. Type the closing one anyway; it will skip over it.

What just happened?

Colab read your cell from top to bottom. It remembered `shelter_name`, `rabbits`, and `cats`. Then `print` showed the results. Colab keeps those variables until the session ends, which is why a *later* cell can use them too. You do not need to define them again.

Part B: Your Turn

Click **+ Code** to make a new cell. Do these **without copying**:

1. Make a variable for the parrot count. (There is 1 parrot.)
2. Print the total number of animals: rabbits + cats + parrot.
3. Print a welcome message that uses **your own** shelter name.
4. **Bonus:** Make Mia's rescue grow. Add a variable `hamsters = 3` and print the new total.

Hints are in the back of the manual, under **Hints**. Try for 5 minutes first. Answers are under **Check Your Work**.

Break It on Purpose

In your first cell, delete one quote mark from `"Paws Haven"` and run it. Read the red error. What line does it point to? Fix it and run again.

Checkpoint Question

If you rename `cats` to `kittens` in the **first** line, but do not change the `print` line, what will happen? Write your guess, then try it.

Stuck?

- **Nothing happens when I press play.** Wait a few seconds. The first run takes longer because Colab is starting.
- **NameError.** You used a variable from a cell you have not run yet. Run the earlier cell first.
- **Everything disappeared.** The session reset. Re-run your cells from the top (Runtime, then Run all).
- **The play button spins forever.** Click Runtime, then Restart session, and try again.

Explain-Back

In 30 seconds, tell someone: **What is a cell? What does Shift + Enter do? Why can a later cell use a variable from an earlier cell?**

Lesson 1 Log (fill in when you finish)

Date: _____ Minutes spent: _____ Mood (1 = grumpy, 5 = great): ____

Hint rung I reached (1-5): ____ Did I explain it to someone? [ ] yes

The one thing I learned: _____

Lesson 2: The Case of the Crashing Programs

Time: about 35 minutes. Goal: read an error message like a detective.

The Story

It is Monday morning at Paws Haven. Mia's helper program has crashed, and so has the one that feeds the cats. The screen is full of red text. Mia could panic... or she could do what real programmers do: **read the clue**.

Error messages are not scary. They are the computer telling you *exactly* where to look. Today you become a bug detective.

The Detective Method

3 steps. Use them every time.

1. **Read the LAST line first.** It names the type of error and says what is wrong.
2. **Find the line number.** Colab shows an arrow or a line number near the top of the red text.
3. **Look at that line, and the line above it.** The mistake is often just before the place Python complains.

Part A: Five Crash Reports

Type each program into its own cell and run it. After each one, use the detective method. The error text in the green boxes may look slightly different in your Colab, but the last line will be close.

Case 1: The missing animal

```
cats = 7
rabits = 12
print("Total:", cats + rabbits)
```

Predict: Will it work?

```
NameError: name 'rabbits' is not defined
```

Python is saying: *I have never heard of a thing called* `rabbits`. Look closely: the line above spells it `rabits`. A **NameError** usually means a spelling mistake, or a variable you forgot to create or run.

Case 2: Words and numbers do not mix

```
name = "Luna"
age = 3
print("Luna is " + age + " years old")
```

```
TypeError: can only concatenate str (not "int") to str
```

The `+` sign can glue words to words, or add numbers to numbers, but it cannot glue a number into a sentence. Two fixes: use commas like `print("Luna is", age, "years old")`, or turn the number into text with `str(age)`. A **TypeError** means: you used the wrong *type* of thing.

Case 3: The list that is too short

```
animals = ["cat", "rabbit", "parrot"]
print(animals[3])

IndexError: list index out of range
```

Lists count from **0**. So this list has spots 0, 1, and 2. There is no spot 3. An **IndexError** means you asked for a spot that does not exist. Fix: `animals[2]` (or `animals[-1]` for the last one).

Case 4: The loop that forgot to be indented

```
for animal in ["cat", "rabbit", "parrot"]:
print(animal)

IndentationError: expected an indented block after 'for' statement on line 1
```

Python uses spaces at the start of a line to know what is *inside* the loop. Put 4 spaces before `print(animal)`. This one is easy to see because Python tells you the line.

Case 5: The sneaky one (no red text!)

```
cats = 7
rabbits = 12
average = cats + rabbits / 2
print("Average per kind:", average)
```

Predict: What should the average of 7 and 12 be? What do you think it prints?

```
Average per kind: 13.0
```

There is **no error** at all, but the answer is wrong! The average of 7 and 12 is 9.5. Python does division *before* addition, so it calculated `12 / 2` first, then added 7. The fix is brackets: `(cats + rabbits) / 2`. This is called a **logic bug**. The computer did exactly what you said, not what you meant. These are the hardest bugs, because nothing warns you. The only defense is to **predict the answer first** and notice when it does not match.

Error cheat sheet

Error name	Usually means
NameError	Misspelled variable, or you forgot to create or run it
SyntaxError	Missing quote, bracket, colon, or comma
IndentationError	Spaces at the start of the line are wrong
TypeError	Wrong type (for example, mixing words and numbers)
IndexError	You asked for a list spot that does not exist
KeyError	You asked for a dictionary or table name that does not exist
AttributeError	That object does not have the action you called

Part B: Your Turn - Fix These Four

Type each program, run it, and use the detective method. Write down (1) the error name, (2) the line, (3) your fix. One of them has no error message. Can you tell which?

Case A

```
dogs = 4
puppies = 3
print("Total dogs:", dogs + puppys)
```

Case B

```
animals = ["cat", "rabbit", "parrot"]
for animal in animals
    print("Hello,", animal)
```

Case C

```
weights = [4, 2, 6]
total = weights[0] + weights[1] + weights[2]
average = total / 2
print("Average weight:", average)
```

Case D

```
name = "Kiwi"
print("Name:" name)
```

Part C: Bug Maker

Now switch roles. Write a **short working program** (3-5 lines) about animals. Then plant **one bug** in it. Give it to someone at home and see if they can find and fix it. Planting bugs teaches you how bugs are born.

Break It on Purpose

Make up your own error on purpose. Which error name can you make appear that is not in the cheat sheet? (Try `print(1/0)` or `int('hello')`.)

Checkpoint Question

Mia's program prints `Total: 15` but she expected 20, and there is no red text. Which of the five cases does this look like, and what is your first move?

Stuck?

- **I fixed one error and a new one appeared.** That is normal! Fixing the first error lets Python get further and find the next one. Keep going.
- **The error line number looks wrong.** Look at the line above it too. Python reports where it *noticed* the problem, which can be just after the mistake.
- **The red text is very long.** Ignore most of it. Read only the last line, then find the arrow line.

Explain-Back

In 30 seconds, tell someone the 3 steps of the detective method, and explain what a **logic bug** is.

Lesson 2 Log (fill in when you finish)

Date: _____ Minutes spent: _____ Mood (1 = grumpy, 5 = great): ____

Hint rung I reached (1-5): ____ Did I explain it to someone? [ ] yes

The one thing I learned: _____

Lesson 3: From Lists to Tables

Time: about 45 minutes. Goal: turn a list of animals into a table and ask it questions.

The Story

Mia keeps her animal records in a paper notebook: name, kind, weight, and age. When she wants to know "What is the average weight?" or "Which cats are older than 3?" she has to count by hand. That takes forever.

Today she types the records into Python. A tool called **pandas** turns her list into a **table** that can answer questions in one line. Almost every machine learning project starts with a table like this, so this is a big skill.

Words to know

Word	Meaning
Row	One animal (one line across the table).
Column	One kind of information, like weight (one line down the table).
DataFrame	pandas' name for a table.
Index	The row numbers on the left, starting at 0.

Part A: Copy and Run

Type this into a new cell. You already know lists and dictionaries; this is a **list of dictionaries**.

```
import pandas as pd

animals = [
    {"name": "Luna",    "kind": "cat",    "weight_kg": 4.0, "age": 3},
    {"name": "Biscuit", "kind": "rabbit", "weight_kg": 2.0, "age": 2},
    {"name": "Kiwi",    "kind": "parrot", "weight_kg": 0.4, "age": 5},
    {"name": "Shadow",  "kind": "cat",    "weight_kg": 5.0, "age": 6},
    {"name": "Clover",  "kind": "rabbit", "weight_kg": 1.8, "age": 1},
]

table = pd.DataFrame(animals)
print(table)
```

Predict: How many rows and how many columns will the table have?

You should see (spacing may differ a little):

```
      name    kind  weight_kg  age
0     Luna     cat        4.0    3
1  Biscuit  rabbit        2.0    2
2     Kiwi  parrot        0.4    5
3   Shadow     cat        5.0    6
4   Clover  rabbit        1.8    1
```

Each dictionary became a row. Each key became a column name. Now ask the table questions. Run each of these in its own cell, and predict first:

```
print(table.shape)                          # rows, columns
print(table["weight_kg"])                   # one column
print(round(table["weight_kg"].mean(), 2))  # average weight
print(table["weight_kg"].max())             # heaviest
```

Try these next. They filter, sort, and count:

```
print(table[table["kind"] == "cat"])          # only the cats
print(table.sort_values("age"))               # youngest first
print(table["kind"].value_counts())           # how many of each kind
```

The first line reads like English: *from the table, take the rows where the kind column equals cat*. The `==` (two equal signs) means "is it equal to?" One equal sign would mean "store this in a variable." In `value_counts()`, animals tied on the same count may appear in either order.

Things to notice

- A table has a **shape**: `(5, 4)` means 5 rows and 4 columns. In machine learning, you will check the shape constantly.
- The dot appears again: `table.sort_values(...)`, `table["weight_kg"].mean()`. In Lesson 4 you will find out why.
- A column like `table["age"]` works like a list of numbers, but with superpowers such as `.mean()` and `.max()`.

What just happened?

You took data you already understood (a list of dictionaries) and gave it to pandas. pandas built a table, and now one short line can answer questions that would have needed a loop. Tables are how computers see data when they learn.

Part B: Your Turn

Time to make a new shelter. Build a **new table** called `zoo` with at least **6 animals of your choice**. Use a different mix of kinds than Mia (for example, dogs, hamsters, turtles). Each animal needs a `name`, `kind`, `weight_kg`, and `age`. Then, without copying:

1. Print the whole table.
2. Print how many rows and columns it has.
3. Print the **average age**, rounded to 1 decimal place.

4. Print only the animals **heavier than 3 kg**.
5. Print the table sorted so the **heaviest animal is first**. (Hint: look up what `ascending=False` means in the hints.)
6. Print how many animals there are of each kind.

Bonus: Print only the animals that are **both** older than 2 **and** lighter than 5 kg. (This one is tricky. Check the hints after trying for 10 minutes.)

Break It on Purpose

Run `print(table["Weight_kg"])` with a capital W. Read the error. What is the error name, and what does it say about the name you asked for?

Checkpoint Question

Looking at Mia's table, what will `print(table[table["age"] > 3])` show? Write down which animals you expect, then run it.

Stuck?

- **NameError: name 'pd' is not defined.** The `import pandas as pd` cell did not run. Run it first.
- **KeyError.** A column name is spelled wrong, or has the wrong capital letters. Print `table.columns` to see the real names.
- **My table looks different.** Compare your list with the one in the manual. A missing comma or bracket changes everything.
- **I get a long red error about `==`.** Make sure you used two equal signs inside the square brackets.
- **The table is not showing nicely.** In Colab you can write just `table` on the last line of a cell instead of `print(table)`; it shows a nicer table.

Explain-Back

In 30 seconds, tell someone: **What is a row? What is a column? How do you ask a table for only the cats?**

Lesson 3 Log (fill in when you finish)

Date: _____ Minutes spent: _____ Mood (1 = grumpy, 5 = great): ____

Hint rung I reached (1-5): ____ Did I explain it to someone? [ ] yes

The one thing I learned: _____

Lesson 4: Things with a Dot

Time: about 35 minutes. Goal: understand objects well enough to use them.

The Story

Mia built a helper robot named **Bolt**. Bolt knows its name and how much battery it has left. It can also *do* things: feed an animal, or recharge itself.

In programming, a thing that holds information *and* can do actions is called an **object**. You write `bolt.feed("Luna")`: the thing, then a dot, then the action. You have already used this in Lesson 3: `table.sort_values(...)`. The table is an object, and `sort_values` is an action it can do.

The three words you need

Word	What it means	Example
Object	A thing with information inside and actions it can do	a table, a robot, a model
Method	An action an object can do (it is a function that belongs to the object)	table.sort_values(), bolt.feed()
Attribute	A piece of information stored inside the object	bolt.battery, bolt.name

Part A: Copy and Run

First, objects you already have. Type and run:

```
word = "rabbit"
print(word.upper())
print(word.count("b"))

animals = ["cat", "rabbit"]
animals.append("parrot")
print(animals)
```

Predict: What does each print do?

```
RABBIT
2
['cat', 'rabbit', 'parrot']
```

The text `"rabbit"` is an object, and `.upper()` and `.count()` are actions it can do. The list is an object too, with `.append()`. Now let us make our own object type. Type this into a new cell, **exactly**, including the spaces at the start of lines:

```
class Robot:
    def __init__(self, name):
        self.name = name
        self.battery = 100

    def feed(self, animal):
        self.battery = self.battery - 10
        print(self.name, "fed", animal, "- battery now", self.battery)

bolt = Robot("Bolt")
bolt.feed("Luna")
bolt.feed("Kiwi")
print(bolt.battery)
```

Predict: What is the last number printed?

```
Bolt fed Luna - battery now 90
Bolt fed Kiwi - battery now 80
80
```

What just happened?

`class Robot` is the **blueprint**, like a recipe. It says every robot has a name, starts with 100 battery, and can feed animals. `bolt = Robot("Bolt")` builds one real robot from the blueprint. That robot is an **object**. `self` just means "this particular robot". When Bolt feeds an animal, `self.battery` is Bolt's battery, not anyone else's. `__init__` runs once when the robot is built, to set up its starting information.

> **Good news: you do not need to write classes yet**
>
> Most of the time you will **use** objects other people built, not write your own blueprints. Soon you will see lines like `model.fit(data)` and `model.predict(new_animal)`.
>
> Now you can read them: `model` is an object, and `fit` and `predict` are actions it knows how to do. That is all you need to know for now. You will write your own classes later, if a project needs them.

Part B: Your Turn

1. Create a second robot named `"Spark"`. Have Spark feed `"Biscuit"` once. Print Spark's battery **and** Bolt's battery. Are they the same?
2. Add a new method to the class called `recharge` that sets the battery back to 100 and prints a message like `Bolt is fully charged!`.
3. **Important:** after you change the class, re-run the class cell, then re-run the lines that create `bolt` and `spark`. Then call `bolt.recharge()` and print the battery.
4. **Bonus:** Change `feed` so that if the battery is 0 or lower, it prints "needs charging!" instead of feeding. Then make a robot feed 11 animals in a row. (A loop will help.)

Break It on Purpose

Run `bolt.dance()` (before you add any dance method). Read the error. What is the error name? What does it tell you about what Bolt can and cannot do?

Checkpoint Question

If Spark feeds 3 animals, does **Bolt's** battery change? Why or why not? Write your guess, then test it.

Stuck?

- **IndentationError.** Everything inside the class needs 4 spaces, and everything inside a method needs 8. Compare your cell to the manual line by line.
- **TypeError about "positional arguments".** Every method needs `self` as its first name, even though you do not type it when calling.
- **AttributeError: ... has no attribute 'recharge'.** You added the method but did not re-run the class cell, or you are still using the old `bolt`. Re-run the class cell, then re-create the robots.
- **I do not get what `self` is.** That is fine. For now: it means "this robot". It will click after you make Spark.

Explain-Back

In 30 seconds, tell someone: **What is an object? What is the difference between a class and an object? What does the dot do?**

Lesson 4 Log (fill in when you finish)

Date: _____ Minutes spent: _____ Mood (1 = grumpy, 5 = great): ____

Hint rung I reached (1-5): ____ Did I explain it to someone? [ ] yes

The one thing I learned: _____

Lesson 5: The Paws Haven Report (Your Big Project)

Time: about 60-90 minutes (you may split it over two sessions). Goal: build a small data project from a blank page.

The Story

The mayor is coming to visit Paws Haven tomorrow and Mia wants to impress her with a **report**: a table of animals, a few facts about them, and a chart.

Mia has no time. She needs **you** to build it. There is no code to copy this time. You have a spec (a list of what the project must do) and everything you have learned. This is how real programmers work.

One new tool: drawing a chart

You have not made a chart yet, so here is one example. Run it after the table from Lesson 3 exists:

```
import matplotlib.pyplot as plt

table["kind"].value_counts().plot(kind="bar")
plt.title("Animals at Paws Haven")
plt.xlabel("Kind")
plt.ylabel("How many")
plt.show()
```

You should see a bar chart with one bar per kind of animal. `plt` is short for the plotting tool, and each `plt.something()` line is an action on the chart. You know the dot now: this is the same idea.

The Spec

Your project must...

1. Use a **theme you like**. It can be a different animal shelter, a zoo, your pets, wild animals, or even your favorite video-game creatures.

2. Start with a **list of dictionaries** with at least **8 items** and at least **4 keys**. At least one key must be a **number** (like weight or age) and one must be a **category** (like kind or habitat).

3. Turn the list into a pandas table and print it.

4. Print the table's **shape**.

5. Print the **average** and the **maximum** of one number column.

6. Print how many items are in **each category**.

7. Print a **filtered** table (for example, only items above a certain number).

8. Print a **sorted** table.

9. Draw **one bar chart** with a title and labels on both axes.

10. Write **at least 3 comments** (lines starting with #) that explain what your code does.

11. Add a **text cell** at the top (+ Text) with the project title and one sentence about what the project is.

Your Checklist

Done?	Requirement
[ ]	Theme chosen, 8+ items, 4+ keys (1 number, 1 category)
[ ]	Table printed, shape printed
[ ]	Average and maximum printed
[ ]	Count per category printed
[ ]	Filtered table printed
[ ]	Sorted table printed
[ ]	Bar chart with title and axis labels
[ ]	At least 3 comments and a title text cell
[ ]	Runtime, then Run all, works with no red errors

How to build it (plan before you code)

1. On paper, write your theme and the 4 keys you will use.

2. Write the first 3 items on paper. Then type the list into Colab.
3. Build **one feature at a time**. After each one, run it and check that it works before moving on. Tick the checklist.
4. When something breaks: Detective Method first (Lesson 2), then the hint ladder.

You may look at your old lessons for how to do things. That is what real programmers do too. But do **not copy and paste the code** from them. Retype it with your own names and your own data. If you find yourself stuck on a step, check the hints at the back.

Part C: The No-Manual Challenge

When the project works, **close this manual**. Open a fresh notebook. In 10 minutes, with no help, do this from memory:

1. Make a list of 3 dictionaries about anything you like.
2. Turn it into a table.
3. Print the average of one number column.

It is okay if it takes more than one try. If you get stuck, note **exactly which step** and tell your coach. That tells us what to practice more. If you can do it, you have really learned this, and you are no longer copying.

Your Graduation Gate

You are ready for Manual 1 when...

- All 10 checklist boxes are ticked and **Run all** works with no red errors.
- You used hint rung 4 or 5 on **at most one** thing.
- You explained your project to someone, in your own words, in 1 minute.
- You tried the No-Manual Challenge.
- You filled in your Project Shelf entry (next page).

Stuck?

- **I do not know what theme to pick.** Pick the animals you like most. You can also use sports teams, Pokemon, or planets. The code works the same.
- **My chart is empty or does not appear.** Make sure the last line of the cell is `plt.show()` and that the column name is spelled exactly right.
- **I get a KeyError.** Print `table.columns` to see the real column names, then check your spelling and capital letters.
- **The filter gives an empty table.** Your condition may be too strict. For example, if every animal is under 10 kg, then `> 10` matches nothing. That is not an error. Try a smaller number.
- **It worked yesterday and now it does not.** The session reset. Use Runtime, then Run all.

Explain-Back

Present your project to someone like the mayor is visiting. Tell them: **What is it? What data did you use? What did you find? What was the hardest bug?**

Lesson 5 Log (fill in when you finish)

Date: _____ Minutes spent: _____ Mood (1 = grumpy, 5 = great): ____

Hint rung I reached (1-5): ____ Did I explain it to someone? [ ] yes

The one thing I learned: _____

Your Project Shelf

Every finished project gets a spot on the shelf. In a few months you will look back and see how much you have built. Take a screenshot of your best chart and keep it in a folder with your notebook.

Shelf Entry: Manual 0

Project name: _____

What it does (1 sentence): _____

_____

Date finished: _____ Hardest bug I fixed: _____

I am proudest of: _____

Screenshot saved? [ ] yes

What you can do now

Tick each one that you can honestly do **without looking**:

- [ ] Open Colab, run a cell, and fix a session reset.
- [ ] Read the last line of an error and say what it means.
- [ ] Spot a logic bug by predicting the answer first.
- [ ] Turn a list of dictionaries into a pandas table.
- [ ] Filter, sort, count, and average a table column.
- [ ] Draw a bar chart with a title and labels.
- [ ] Explain what an object, a method, and an attribute are.
- [ ] Build a small project from a spec, one feature at a time.

Progress Report (for the Coach)

Fill in this page as the lessons go. When you send me an update, include a photo or copy of this table plus any errors that stuck your child. The highest hint rung is the most important number: it shows how independent the work was.

Lesson	Date	Minutes	Highest hint rung (1-5)	Explain-back worked? (Y/N)	Confidence (1-5)	Notes (errors that stuck, wins)
1 Control Room						
2 Crashing Programs						
3 Lists to Tables						
4 Things with a Dot						
5 Big Project						
No-Manual Challenge						

What I need from you when you report

- This table (photo is fine).
- The final project code from Lesson 5 (copy the cells into a message).
- Anything your child said, in their own words, about what was hard or fun.
- Whether the pace (about 2 hours a week) felt right: too slow, right, or too fast.

What I will do with it

I will check whether the child is ready for Manual 1 (the first real machine learning project: teaching a program to tell cats from rabbits), and adjust the pace, the amount of copying, and the difficulty to fit what the child actually did.

Hints

Read only the hint for the part you are stuck on, and only after 10 minutes of trying.

Lesson 1, Part B

- **Hint 1:** Make a variable the same way as the others, like `parrots = 1`.
- **Hint 2:** You can add three numbers inside `print`, just like the example added two.

Lesson 2, Part B

- **Case A: Hint 1:** Compare the spelling of the variable on line 2 with the one in the print. **Hint 2:** One letter is different.
- **Case B: Hint 1:** Look at the end of the first line. Compare it with the loop in Case 4. **Hint 2:** Python needs a special symbol at the end of a `for` line.
- **Case C: Hint 1:** The program does not crash. Work out the correct average by hand first. **Hint 2:** How many numbers are in the list?
- **Case D: Hint 1:** Read the error out loud. It might even suggest something. **Hint 2:** What goes between two things you print?

Lesson 3, Part B

- **Hint 1:** Copy the *shape* of Mia's list, but change every name and number. Do not forget the commas.
- **Hint 2 (sorting):** `zoo.sort_values("weight_kg", ascending=False)` puts the biggest value first. Normally it goes smallest first.
- **Hint (rounding):** `round(number, 1)` keeps 1 decimal place.
- **Hint (bonus):** Use two conditions with `&` between them, and put each one in brackets: `zoo[(zoo["age"] > 2) & (zoo["weight_kg"] < 5)]`.

Lesson 4, Part B

- **Hint 1:** `spark = Robot("Spark")` builds a second robot. It has its own battery.
- **Hint 2 (recharge):** A new method is a new `def` inside the class, with `self` as its first name. Look at how `feed` is written.
- **Hint (bonus):** Use `if self.battery <= 0:` and `else:` inside `feed`.

Lesson 5, Project

- **Stuck on the list:** Write one dictionary first. Copy and change it to make the others.
- **Stuck on the chart:** Use the chart example from this lesson, but change the table and the column names.
- **Stuck on filtering:** Look at Lesson 3's `table[table["kind"] == "cat"]`. Change the column name, the symbol, and the value.

- **Stuck on where to start:** Do just the table and the shape. Run it. Then add one feature at a time.

Check Your Work

Only look here after you have finished your own attempt. Your answers may be different and still be right.

Lesson 1

Checkpoint: You get a **NameError: name 'cats' is not defined**, because the `print` line still uses the old name `cats`, which no longer exists.

Part B (one possible answer):

```
parrots = 1
print("Total animals:", rabbits + cats + parrots)
print("Welcome to Sunny Paws!")
hamsters = 3
print("New total:", rabbits + cats + parrots + hamsters)
```

Remember: this needs the first cell to have been run, so `rabbits` and `cats` exist.

Lesson 2

Part B:

Case	Error	Fix
A	NameError: name 'puppys' is not defined	Spell it `puppies`
B	SyntaxError: expected ':'	Add a colon: `for animal in animals:`
C	No error! Logic bug. It prints 6.0 but the true average is 4.0.	Divide by 3, or use `len(weights)`
D	SyntaxError: invalid syntax (maybe a missing comma)	Add a comma: `print("Name:", name)`

Checkpoint: It looks like Case 5, a logic bug. The first move is to work out the right answer by hand, then look at the line that does the maths and check the order of operations (brackets!).

Lesson 3

Checkpoint: It shows the rows where age is greater than 3: **Kiwi (5)** and **Shadow (6)**.

Part B (example):

```
import pandas as pd

zoo_list = [
    {"name": "Rex",    "kind": "dog",     "weight_kg": 12.0, "age": 4},
    {"name": "Pip",    "kind": "hamster", "weight_kg": 0.1,  "age": 1},
    {"name": "Shelly", "kind": "turtle",  "weight_kg": 3.5,  "age": 20},
    {"name": "Buddy",  "kind": "dog",     "weight_kg": 9.0,  "age": 2},
    {"name": "Nibbles","kind": "hamster", "weight_kg": 0.12, "age": 2},
    {"name": "Flash",  "kind": "turtle",  "weight_kg": 2.8,  "age": 15},
]
zoo = pd.DataFrame(zoo_list)

print(zoo)
print(zoo.shape)
print(round(zoo["age"].mean(), 1))
print(zoo[zoo["weight_kg"] > 3])
print(zoo.sort_values("weight_kg", ascending=False))
print(zoo["kind"].value_counts())
```

Bonus: `print(zoo[(zoo["age"] > 2) & (zoo["weight_kg"] < 5)])`. Each condition needs its own brackets, and `&` means "and".

Lesson 4

Checkpoint: No. Each robot is its own object with its own battery. When Spark's `self.battery` changes, Bolt's does not, because `self` means "this one robot."

Part B (the changed class):

```
class Robot:
    def __init__(self, name):
        self.name = name
        self.battery = 100

    def feed(self, animal):
        if self.battery <= 0:
            print(self.name, "needs charging!")
        else:
            self.battery = self.battery - 10
            print(self.name, "fed", animal, "- battery now", self.battery)

    def recharge(self):
        self.battery = 100
        print(self.name, "is fully charged!")

bolt = Robot("Bolt")
spark = Robot("Spark")
spark.feed("Biscuit")
print(spark.battery, bolt.battery)    # 90 100
bolt.recharge()

for i in range(11):
    spark.feed("Clover")
```

The last loop prints "needs charging!" once the battery is used up. Notice that after the 9th feeding, Spark's battery reaches 0, so feedings 10 and 11 refuse. (Spark already fed Biscuit once, so it had 90 to start with.)

Lesson 5

There is no single right answer. Check your work against the spec on the checklist. A project that satisfies all items and runs with **Run all** is a success, even if the code looks different from anyone else's. Look for: readable names, comments that say *why*, and a chart with a title and axis labels.

Glossary

Word	Plain meaning
Cell	A box in Colab where you write and run code.
Session	Colab's short-term memory. When it resets, variables are forgotten.
Variable	A name that holds a value.
Error / Exception	Python's way of telling you something went wrong.
Traceback	The red text that shows where the error happened.
Bug	A mistake in a program.
Logic bug	A bug that gives a wrong answer without any error message.
pandas	A tool for working with tables of data.
DataFrame	A pandas table: rows and columns.
Row / Column	One item across / one kind of information down.
Shape	How many rows and columns a table has, like (5, 4).
Object	A thing that holds information and can do actions.
Method	An action that belongs to an object, called with a dot.
Attribute	A piece of information stored inside an object.
Class	A blueprint used to build objects.
Comment	A note in your code that starts with # and is ignored by Python.

Coming Next: Manual 1

Mia has a problem. Two animals arrive at Paws Haven in a box, and the label is missing. All Mia has is their weight and ear length. Can a program learn from the animals she already knows and guess what the new ones are? In Manual 1 you will teach a computer to learn from examples. That is machine learning, and you are ready for it.